Bounded Precision-Geometry Scaling for Robust Multi-Task Learning under Loss Scale Mismatch

Abstract

Multi-task learning often combines objectives with different numerical scales, which can destabilize uncertainty-based loss weighting. We propose Bounded Precision-Geometry Scaling (BPGS), a split uncertainty-weighting method that maps each task's latent uncertainty coordinate through a bounded sigmoid chart and, in its batch-aware form, expresses task log-variances in detached batch log-loss coordinates with first-batch calibration. The normalized BPGS weights are exactly invariant to uniform loss rescaling when the numerical safeguards are inactive. We evaluate BPGS on synthetic stress tests, NYUv2 dense prediction, Yeast multi-label classification, and RF1 multi-target regression. Under pure post-hoc loss rescaling, BPGS keeps macro score nearly unchanged from $\times 1$ to $\times 1000$ (0.777 to 0.778), whereas Kendall-style uncertainty weighting drops from 0.780 to 0.637; L1-normalizing Kendall's weights does not recover this stability. On NYUv2, BPGS attains the best depth absolute relative error (0.223), the best depth RMSE (0.790), and the lowest total loss (1.891) among the compared methods, including Nash-MTL. Additional controlled studies find low sensitivity to the tested batch sizes and calibration batches, and negligible overhead relative to Kendall. On Yeast and RF1, BPGS remains competitive beyond synthetic stress settings, achieving the best Yeast micro-F1 (0.616) and competitive RF1 performance without leading the table on RMSE or MAE. These results support BPGS as a scale-robust refinement of uncertainty weighting when loss-scale mismatch is a primary concern.

1 Introduction

Multi-task learning optimizes several objectives with shared parameters [1]. In practice, those objectives often differ not only in semantics but also in numerical scale, curvature, and noise. This mismatch appears in dense prediction, where segmentation, depth, and surface-normal losses coexist, and in tabular settings that combine multiple binary or regression targets. When the weighting rule is sensitive to raw loss magnitude, optimization can become brittle: one task can dominate the shared update, and adaptive uncertainty variables can drift toward uninformative regimes.

Classical homoscedastic uncertainty weighting adapts task weights by learning one log-variance parameter per task [2]. That formulation is effective in many settings, but its uncertainty coordinates are unconstrained, so severe loss-scale mismatch can translate directly into large precision shifts. Methods such as PCGrad [3] address a different failure mode by modifying conflicting gradients rather than stabilizing the scale of learned uncertainty weights. This leaves a practical gap for a method whose primary goal is robustness to heterogeneous task scales without giving up adaptive weighting.

We study Bounded Precision-Geometry Scaling (BPGS), a bounded uncertainty-weighting method designed for that gap. BPGS assigns one learnable uncertainty coordinate to each task, maps it through a bounded sigmoid chart, and in its canonical form converts that bounded latent coordinate into a batch-aware log-variance using detached log-loss statistics. The resulting precisions are used in a split optimization rule: network parameters are updated with detached normalized weights, while

Preprint.

uncertainty coordinates are updated in a separate uncertainty objective. We do not position BPGS as a universal replacement for all multi-task optimizers. The question studied here is narrower: whether bounded, batch-aware uncertainty parameterization improves robustness under scale mismatch while remaining competitive on standard benchmarks.

Our contributions are empirical and methodological:

1. We formulate BPGS as a bounded, batch-aware, split uncertainty-weighting method and prove that its normalized weights are invariant to uniform loss rescaling when the numerical safeguards are inactive.
2. On a pure post-hoc loss-rescaling stress test, BPGS remains effectively invariant from $\times 1$ to $\times 1000$ scaling (macro score 0.777 to 0.778), whereas Kendall-style uncertainty weighting drops from 0.780 to 0.637; L1-normalizing Kendall's weights does not recover this stability.
3. On the main NYUv2 benchmark, BPGS attains the best depth absolute relative error (0.223), the best depth RMSE (0.790), and the lowest total loss (1.891) among the compared methods, including Nash-MTL. Separate NYUv2 ablations isolate the role of the batch-aware chart, first-batch calibration, and split stop-gradient.
4. Controlled NYUv2 diagnostics measure sensitivity to batch size and first-batch calibration, and quantify the runtime and memory overhead relative to Kendall weighting.
5. On Yeast multi-label classification and RF1 multi-target regression, BPGS remains competitive outside synthetic stress settings, achieving the highest Yeast micro-F1 (0.616) while on RF1 it is competitive but not best on RMSE or MAE.

2 Related Work

Multi-task learning is commonly framed as shared-representation learning across related objectives [1]. Within that setting, adaptive weighting methods try to stabilize optimization by changing the scalar contribution of each task. The closest prior work to BPGS is homoscedastic uncertainty weighting, which learns one log-variance per task and uses that variable both to rescale the task loss and to regularize the uncertainty estimate [2]. Other adaptive weighting rules reweight tasks according to training-rate or performance signals, including GradNorm [4] and dynamic task prioritization [5]. Those methods differ from BPGS in what they adapt: GradNorm tunes gradient magnitudes to balance training rates, and dynamic task prioritization emphasizes tasks that are currently worse. BPGS stays in the uncertainty-weighting family, but it changes the geometry of the learned coordinate: the uncertainty variable is mapped through a bounded chart and, in the batch-aware form, anchored to detached batch log-loss statistics. The intended effect is scale robustness, not a new general rule for balancing arbitrary task conflicts.

Another line of work treats multi-task learning as a multi-objective optimization problem and modifies gradients directly. Sener and Koltun cast multi-task learning as Pareto-style optimization and derive a tractable surrogate for deep networks [6]. PCGrad addresses harmful gradient interference by projecting conflicting task gradients [3], and CAGrad regularizes the update toward conflict-averse directions [7]. More recent methods include Nash-MTL, which frames the update as a bargaining game over task gradients [8], as well as IMTL-G [9] and FAMO [10]. These methods target a different failure mode from BPGS: they operate on gradients or solve a per-step multi-objective problem, whereas BPGS changes only the scalar uncertainty weights. Nash-MTL is included in our NYUv2 comparison; PCGrad is included on the tabular tasks. CAGrad, IMTL-G, and FAMO are discussed for positioning but are not evaluated here.

Taken together, the related literature suggests a narrow positioning for BPGS: it is best understood as a scale-robustness-focused refinement of uncertainty weighting, not as a general multi-task optimization advance.

3 Method

3.1 Problem setup

Let T denote the number of tasks, let w denote the shared network parameters, and let

$$L(w) = \big(L_1(w), \ldots, L_T(w)\big) \tag{1}$$

collect the raw task losses for the current batch. The goal is to construct adaptive task weights that remain informative when the task losses differ substantially in scale. BPGS does this by learning one latent uncertainty coordinate per task and mapping that coordinate into a bounded log-variance chart.

The design goal is not to replace task weighting altogether, but to make the weighting rule less fragile when the same task is measured in different numerical units or when one loss is much larger than the others. In that situation, an unconstrained uncertainty parameter can move too far too quickly, so the method deliberately constrains the coordinate and ties it to the current batch's loss statistics.

3.2 Canonical BPGS chart

For the canonical batch-aware variant, BPGS computes detached log-loss statistics

$$\widetilde{L}_i = \max(L_i, \varepsilon_{\log}), \qquad \mu(L) = \frac{1}{T}\sum_{i=1}^{T} \mathrm{sg}[\log \widetilde{L}_i], \tag{2}$$

and

$$\bar{\varsigma}(L) = \max\left(\sqrt{\frac{1}{T}\sum_{i=1}^{T}\big(\mathrm{sg}[\log \widetilde{L}_i] - \mu(L)\big)^2}, \varepsilon_{\mathrm{std}}\right). \tag{3}$$

Each task has a learnable uncertainty coordinate $\theta_i \in \mathbb{R}$. Using the task-count-dependent latent radius

$$\tau_T = \sqrt{T-1} + 0.1, \tag{4}$$

we define the bounded latent coordinate

$$z_i(\theta_i) = \tau_T\big(2\sigma(\theta_i) - 1\big), \tag{5}$$

where $\sigma(\cdot)$ is the logistic sigmoid. The canonical BPGS log-variance is then

$$s_i(\theta_i; L) = \mu(L) + \bar{\varsigma}(L)\, z_i(\theta_i). \tag{6}$$

The radius in Eq. (4) follows from the geometry of a population-standardized T-vector. When the safeguards are inactive, its coordinates have zero mean and unit average square. If one coordinate has magnitude M, the other $T-1$ coordinates must sum to $-M$; their squared sum is minimized when they are equal. Hence $T \geq M^2 + (T-1)(M/(T-1))^2$, giving $M \leq \sqrt{T-1}$. The added 0.1 is a numerical margin that keeps first-batch inverse-sigmoid calibration away from the chart boundary. The corresponding task precision is

$$\omega_i(\theta_i; L) = e^{-s_i(\theta_i; L)}, \tag{7}$$

and the network weight is its L1-normalized version

$$\alpha_i(\theta_i; L) = \frac{\omega_i(\theta_i; L)}{\sum_{j=1}^{T} \omega_j(\theta_j; L)}. \tag{8}$$

The reason for the bounded chart is simple: the uncertainty variable should still adapt, but it should not be free to drift to extreme values just because the raw losses are numerically large or small. Centering the coordinate on the current batch mean and scaling it by the current batch spread makes the representation relative to the batch rather than tied to an absolute loss level. The bounded sigmoid map then keeps the learnable coordinate inside a finite range, which makes the resulting precision easier to control and easier to reason about.

In the canonical configuration used for the main experiments, BPGS performs a one-shot auto-calibration step on the first observed batch. This initialization maps the first batch's detached log-loss statistics into the bounded chart before the first network-weight construction. Ablation variants with fixed initialization or a stateless chart are studied separately, but the primary method studied in this paper is the batch-aware auto-calibrated form above.

This first-batch calibration is a practical choice, not a separate claim of optimality. Its role is to place the uncertainty coordinates in a sensible range before training begins so that the model does not spend the early iterations recovering from an arbitrary starting point. The ablations then test whether that initialization rule matters independently from the bounded, batch-aware chart.

3.3 Split optimization objectives

BPGS uses separate objectives for the network parameters and the uncertainty coordinates. The network objective is

$$J_{\mathrm{net}}(w \mid \theta, L) = \sum_{i=1}^{T} \mathrm{sg}[\alpha_i(\theta_i; L)]\, L_i, \tag{9}$$

where $\mathrm{sg}[\cdot]$ denotes stop-gradient. The uncertainty objective is

$$J_{\mathrm{unc}}(\theta \mid L) = \sum_{i=1}^{T} \left[\frac{1}{2} \omega_i(\theta_i; L)\, \mathrm{sg}[L_i] + \frac{1}{2} s_i(\theta_i; L) \right]. \tag{10}$$

Equation (10) has the same algebraic form as the scalar uncertainty objective of classical homoscedastic uncertainty weighting [2], but it is not the same optimization problem. Kendall's log-variance is unconstrained, whereas BPGS confines s_i to a batch-conditional interval; their fixed-point structures therefore differ even though the objective forms coincide.

The implemented uncertainty update uses a fixed gradient scale $g_\theta = 100$. This constant is used across the reported BPGS configurations to compensate for the smaller uncertainty-gradient magnitudes; it changes the update speed, not the weighting rule. Appendix A lists this and the remaining optimizer settings.

The split is important because the two parameter groups play different roles. The network should see the current task weights as fixed coefficients for the main update, while the uncertainty coordinates should react to the current losses without directly reshaping the network gradient in the same step. Separating the passes makes that division explicit and avoids conflating "how to update the model" with "how to update the weights."

At each iteration, BPGS uses the same current-batch losses for both passes. The network parameters are updated with J_{net}, and the uncertainty coordinates are updated in a separate pass with J_{unc}. In the implemented method, the network pass uses detached normalized weights, while the uncertainty pass treats both raw losses and batch statistics as detached quantities.

In plain language, the network pass answers: given the current task weights, how should the shared model move? The uncertainty pass answers: given the current batch losses, how should the weights be reshaped for the next step? Keeping those questions separate is the core reason the method uses two objectives rather than a single coupled update.

3.4 Batch-conditional boundedness

The key formal property used in our robustness framing is conditional boundedness with respect to the current batch. For any fixed batch L and any finite θ_i,

$$\mu(L) - \bar{\varsigma}(L)\tau_T < s_i(\theta_i; L) < \mu(L) + \bar{\varsigma}(L)\tau_T. \tag{11}$$

This follows immediately from $\sigma(\theta_i) \in (0, 1)$, which implies

$$z_i(\theta_i) \in (-\tau_T, \tau_T). \tag{12}$$

Substituting this interval into the affine map defining s_i gives the stated bound. Consequently, for any fixed batch, the induced precision $\omega_i(\theta_i; L)$ is strictly positive and remains inside a finite interval. The claim is therefore batch-conditional rather than globally independent of the observed loss statistics.

The point of stating this property explicitly is not that boundedness alone proves the method is best, but that it explains why the learned weights stay numerically well-behaved. If the coordinate can only move inside a finite interval for a given batch, then the corresponding precision cannot explode or collapse without limit on that batch. That makes the method's robustness claim more concrete: BPGS is designed to keep the uncertainty parameter in a controlled range while still allowing it to adapt to the batch it sees.

Boundedness alone does not ensure learnability near the boundary: as z_i approaches $\pm\tau_T$, the sigmoid derivative vanishes and recovery can become difficult. We therefore examined the logged main-paper trajectories (Appendix G). The largest observed ratio was $|z_i|/\tau_T = 0.93$ (NYUv2, initialization); the final-epoch caps were at most 0.84 on NYUv2, 0.69 on Yeast, and 0.89 on RF1. The closest

observed point retained a derivative factor of about 0.033 (13% of its maximum), and the final epochs reached about 0.074. Thus no logged run exhibited boundary locking, although this empirical margin is not a guarantee.

3.5 Uniform rescaling invariance

The batch-aware chart has a narrow but useful algebraic property.

Proposition 1 (Uniform rescaling invariance)**.** *Let $L'_i = cL_i$ for every task and some $c > 0$. If the numerical safeguards $\varepsilon_{\log}$ and $\varepsilon_{\mathrm{std}}$ are inactive on both batches, then $\alpha_i(\theta_i; L') = \alpha_i(\theta_i; L)$ for all i. First-batch auto-calibration also produces the same initial θ_i values under the same rescaling.*

Proof. Uniform rescaling adds $\log c$ to every log-loss. Therefore $\mu(L') = \mu(L) + \log c$ while $\bar{\varsigma}(L') = \bar{\varsigma}(L)$, and $z_i(\theta_i)$ is unchanged. Hence $s_i(\theta_i; L') = s_i(\theta_i; L) + \log c$, so each raw precision is multiplied by the same factor $1/c$, which cancels in the L1 normalization. The standardized coordinates used for first-batch calibration are unchanged by the same additive shift. □

The proposition concerns normalized weights on a fixed batch; it does not claim that an entire optimizer trajectory or final predictive metrics are invariant. The pure-rescaling experiment provides the corresponding empirical diagnostic.

4 Experimental Setup

4.1 Benchmarks

We evaluate BPGS across benchmark and stress settings that serve different roles in the paper. First, we use the standard NYUv2 multi-task benchmark [11] for the main dense-prediction comparison. The benchmark contains 795 training images and 654 validation images, with three tasks: semantic segmentation, depth estimation, and surface-normal prediction. The main NYUv2 comparison uses a 120-epoch budget and reports final-epoch metrics aggregated over seeds.

Second, we run a separate NYUv2 ablation study on a fixed 50% training subset while keeping the validation split unchanged. This study isolates the role of the bounded chart and the initialization rule by comparing four BPGS variants against Kendall uncertainty weighting: the canonical batch-aware auto-calibrated form, a batch-aware fixed-initialization form, a stateless fixed-initialization form, and a stateless auto-calibrated form. The ablation runs use a 60-epoch budget and the same three training seeds.

Third and fourth, we evaluate cross-regime generalization on two real-data multi-output problems. The Yeast benchmark [12] is a 14-label multi-label classification task with 1,500 training examples and 917 validation examples. The RF1 benchmark [13] is an 8-target regression task with 4,108 training examples and 5,017 validation examples. In both cases, the tables report final task metrics aggregated over three seeds from runs scheduled for up to 120 epochs.

We also conduct four controlled NYUv2 studies on the fixed 50% training subset used for the ablation. The stop-gradient study compares canonical BPGS with a coupled variant for 60 epochs over three seeds. The batch-size study uses batch sizes 4, 8, and 16, again with three seeds per setting. The first-batch study varies only the loader seed (1001, 2001, 3001) while fixing the model seed. The overhead study compares BPGS and Kendall for 60 epochs over three seeds, recording per-epoch wall-clock time and peak CUDA memory.

Finally, we use three synthetic robustness studies. The pure loss-rescaling study applies post-hoc scale multipliers $\times 1$, $\times 10$, $\times 100$, and $\times 1000$ to test invariance to raw loss magnitude. The scale-stress study uses the same scale grid in a synthetic multi-task setting designed to probe degradation under scale imbalance. The heterogeneous mixed-stress study evaluates three regimes, denoted *clean*, *conflict*, and *noisy*, to test robustness when tasks differ not only in scale but also in interaction structure.

To test whether normalization alone explains the rescaling result, we additionally compare BPGS, Kendall, and Kendall with L1-normalized precision weights on the pure-rescaling grid. This completed diagnostic uses seeds 42, 123, and 999, so it is reported separately from the main rescaling table rather than as a seed-matched replacement.

4.2 Compared methods and metrics

The exact comparison set depends on the benchmark. The main NYUv2 benchmark compares BPGS against Static, Kendall uncertainty weighting [2], UWSO, and Nash-MTL [8]. The NYUv2 ablation compares the four BPGS variants against Kendall. The Yeast and RF1 benchmarks compare BPGS against Static, Kendall, PCGrad [3], a GradNorm-inspired gradient-norm proxy [4], and UWSO. The pure loss-rescaling and scale-stress studies compare BPGS, Kendall, and UWSO, while the heterogeneous mixed-stress study compares BPGS, Kendall, PCGrad, and UWSO.

For NYUv2, we report segmentation mIoU, depth absolute relative error, depth RMSE, mean angular error for normals, normal accuracy within 11.25°, total validation loss, and in the main benchmark also Δ_M against the Static baseline [14]. For Yeast, we report macro-F1, micro-F1, and Hamming accuracy. For RF1, we report R^2, RMSE, and MAE. For the synthetic stress studies, the primary statistic is macro score, with worst-task score used in the heterogeneous analysis and the stress diagnostics.

4.3 Reporting protocol

Unless stated otherwise, values are reported as mean $\pm$ standard deviation across seeds. For the main NYUv2, Yeast, and RF1 tables, and the rescaling and heterogeneous stress studies, we aggregate over seeds 42, 43, and 44. For the synthetic scale-stress study in Table 5, we reran the grid with 10 seeds to strengthen the statistical comparison against Kendall. For the synthetic stress studies, we aggregate per-seed summary statistics. For NYUv2, Yeast, and RF1, we report the final recorded epoch of each run. The anonymized supplementary materials contain the code, configurations, and reproduction instructions.

One reporting rule is important for fairness on NYUv2. Checkpoint summaries are not uniformly available across methods, so mixing best-checkpoint and final-checkpoint numbers would introduce method-dependent extraction rules. We therefore compute all main NYUv2 results from final-epoch metrics for every method. The real-data tables are also reported from final task metrics. Final NYUv2 benchmark runs select checkpoints by `val/miou` in max mode, whereas the ablation and controlled studies use the dataset default `val/total_loss` in min mode; in all cases, the paper tables use the final recorded epoch. The uncertainty update uses $g_\theta = 100$; full configuration details are in Appendix A.

5 Results

We organize the evaluation around four questions: whether BPGS is robust to explicit scale perturbations, whether that robustness transfers to a standard dense-prediction benchmark, which parts of the method matter in ablation, and whether the method remains competitive outside the synthetic stress setting.

5.1 Robustness to loss-scale mismatch

Figure 1 separates two perturbation types. The right column isolates pure post-hoc loss rescaling, where only the numerical scale changes. In that setting, BPGS is effectively invariant: its macro score stays at 0.777, 0.784, 0.781, and 0.778 from $\times 1$ through $\times 1000$ (Table 1 and Appendix Figure 3). Proposition 1 shows that the normalized weights themselves are invariant under this rescaling when the safeguards are inactive; the table is an empirical diagnostic of the resulting training behavior. Kendall-style uncertainty weighting starts slightly higher at $\times 1$ but drops monotonically to 0.637 at $\times 1000$, while UWSO improves modestly from a much weaker starting point.

Normalization is not sufficient. On a separate three-seed diagnostic, L1-normalized Kendall improves over unnormalized Kendall at $\times 1000$ (0.689 versus 0.658) but still falls by 0.105 from $\times 1$ to $\times 1000$; BPGS falls by 0.004 (Appendix B). The completed diagnostic uses a different seed set from the main rescaling table, so this comparison is interpreted as evidence that normalization alone is insufficient, not as a seed-matched estimate of the main-table gap.

The left column of Figure 1 is the harder test because the perturbation changes the optimization problem rather than only its units. All three methods degrade as the factor increases, but BPGS is

Table 1: Pure loss rescaling: Macro Score across scale factors. Mean ± std over 3 seeds.

Method	×1	×10	×100	×1000
Kendall	$\mathbf{0.780 \pm 0.007}$	0.769 ± 0.013	0.714 ± 0.017	0.637 ± 0.015
UWSO	0.622 ± 0.073	0.674 ± 0.036	0.688 ± 0.035	0.681 ± 0.014
BPGS	0.777 ± 0.012	$\mathbf{0.784 \pm 0.008}$	$\mathbf{0.781 \pm 0.001}$	$\mathbf{0.778 \pm 0.006}$

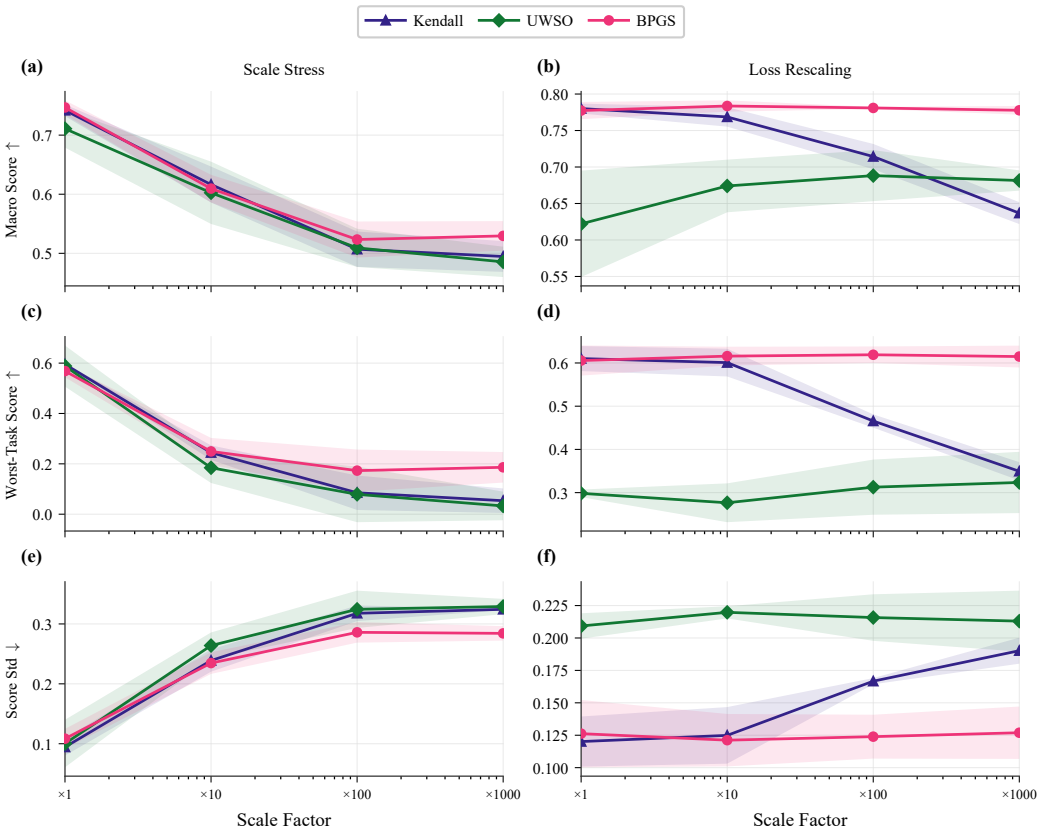

Figure 1: Synthetic robustness diagnostics under scale mismatch. Left column: scale stress, where performance is evaluated as task scales become increasingly imbalanced. Right column: pure loss rescaling, where only the reported magnitude changes. BPGS remains nearly invariant under pure rescaling and retains the strongest macro score at the largest scale-stress factors.

best at ×1, ×10, ×100, and ×1000, reaching 0.543 at ×1000 versus 0.499 for Kendall and 0.494 for UWSO. The relative drop from ×1 to ×1000 is 27.0% for BPGS, compared with 32.3% for Kendall and 28.5% for UWSO (Appendix Table 6; see also Appendix Figure 2). The BPGS–Kendall gap at ×1000 is 0.043, or 1.07 combined standard deviations, so the updated 10-seed estimate is directionally stronger than the original 3-seed result but still should be read as a comparison of means rather than a formal significance test. In our experiments, this provides the strongest support for the bounded, batch-aware construction.

The heterogeneous mixed-stress study is less favorable to BPGS than the explicit scale-mismatch studies (Appendix Table 7 and Appendix Figure 5). Kendall and PCGrad attain higher macro scores in the noisy and conflict regimes, whereas BPGS gives the strongest worst-task score in the clean regime at 0.301 and remains far above the UWSO worst-task score of 0 throughout. We therefore interpret BPGS as a method targeted at loss-scale robustness rather than as a general solution to heterogeneous task interaction.

Table 2: NYUv2 dense prediction benchmark. All methods trained for 120 epochs. Final-epoch metrics reported as mean $\pm$ std over 3 seeds. Δ_M is computed against the Static baseline; the Nash-MTL value is derived from the reported mean metrics and is shown approximately.

Method	mIoU ↑	Abs Rel ↓	RMSE ↓	Angle ↓	Within 11.25° ↑	Total Loss ↓	Δ_M ↑
Static	$\mathbf{0.341 \pm 0.006}$	0.239 ± 0.000	0.829 ± 0.011	35.934 ± 2.055	0.169 ± 0.009	1.992 ± 0.033	0.000 ± 0.000
Kendall	0.281 ± 0.013	0.229 ± 0.001	$\underline{0.808 \pm 0.003}$	28.629 ± 0.431	0.231 ± 0.007	1.977 ± 0.029	0.022 ± 0.000
UWSO	0.294 ± 0.007	$\underline{0.228 \pm 0.003}$	0.815 ± 0.012	$\mathbf{26.110 \pm 0.383}$	$\mathbf{0.271 \pm 0.007}$	$\underline{1.936 \pm 0.023}$	$\underline{0.059 \pm 0.000}$
BPGS	$\underline{0.312 \pm 0.001}$	$\mathbf{0.223 \pm 0.002}$	$\mathbf{0.790 \pm 0.010}$	$\underline{26.851 \pm 0.334}$	$\underline{0.257 \pm 0.005}$	$\mathbf{1.891 \pm 0.014}$	$\mathbf{0.078 \pm 0.000}$
Nash-MTL	0.252 ± 0.027	0.242 ± 0.009	0.856 ± 0.029	30.180 ± 1.010	0.211 ± 0.011	2.071 ± 0.070	≈ -0.038

Table 3: Ablation study: BPGS variants vs. Kendall uncertainty weighting on NYUv2. All methods trained for 60 epochs. Mean $\pm$ std over 3 seeds.

Method	mIoU ↑	Abs Rel ↓	RMSE ↓	Angle ↓	Within 11.25° ↑	Total Loss ↓
Kendall	$\underline{0.137 \pm 0.006}$	0.294 ± 0.010	$\underline{0.990 \pm 0.016}$	38.681 ± 0.936	0.130 ± 0.005	2.546 ± 0.035
BPGS (canonical)	0.128 ± 0.008	$\mathbf{0.283 \pm 0.005}$	$\mathbf{0.988 \pm 0.043}$	$\mathbf{33.834 \pm 1.068}$	$\mathbf{0.161 \pm 0.015}$	$\mathbf{2.488 \pm 0.075}$
BPGS (BA-fixed)	$\mathbf{0.143 \pm 0.006}$	0.291 ± 0.005	0.994 ± 0.022	39.097 ± 0.478	0.130 ± 0.004	$\underline{2.524 \pm 0.047}$
BPGS (SL-auto)	0.058 ± 0.006	0.331 ± 0.003	1.305 ± 0.015	$\underline{34.661 \pm 1.310}$	$\underline{0.147 \pm 0.024}$	6.142 ± 1.332
BPGS (SL-fixed)	0.120 ± 0.007	$\underline{0.287 \pm 0.006}$	1.013 ± 0.025	35.183 ± 0.963	0.142 ± 0.010	2.566 ± 0.051

5.2 Main NYUv2 benchmark

Table 2 shows that no method dominates every metric. Static achieves the best segmentation mIoU, and UWSO is strongest on the two surface-normal metrics. BPGS, however, gives the strongest depth results and a strong aggregate trade-off by the reported summary metrics: it attains the best absolute relative depth error (0.223), the best depth RMSE (0.790), the lowest total loss (1.891), and the highest Δ_M (0.078). The same trade-off pattern is visible in Appendix Figures 8–11.

Relative to the strongest competing values, BPGS improves absolute relative error from 0.228 under UWSO to 0.223 and RMSE from 0.808 under Kendall to 0.790, while also lowering total loss from 1.936 to 1.891. The NYUv2 result therefore supports a specific claim: the robustness-oriented BPGS parameterization does not have to trade away standard benchmark performance to control scale sensitivity.

Nash-MTL has the weakest mean mIoU (0.252) and largest mIoU standard deviation (0.027) in this comparison, while its within-11.25° accuracy (0.211) exceeds that of Static (0.169). This does not establish a general ranking between the methods; it supplies a recent multi-objective baseline under the same NYUv2 protocol.

5.3 Ablation on chart and initialization

Table 3 isolates the role of the batch-aware chart and the initialization rule. The stateless auto-calibrated variant degrades severely, with mIoU 0.058 and total loss 6.142. This shows that first-batch auto-calibration is not sufficient on its own; in these ablations, the batch-aware chart that re-expresses the uncertainty variables in current log-loss coordinates is necessary for stable behavior. Appendix Figure 6 shows this failure in the validation dynamics, while Appendix Figure 7 shows the contrasting task-weight behavior of the stable variants.

Among the stable variants, the best values split between the two batch-aware forms. The canonical batch-aware BPGS variant gives the best depth absolute relative error, the best depth RMSE, the best mean angular error, the best normal accuracy within 11.25°, and the lowest total loss. The batch-aware fixed-initialization variant gives the best mIoU. By contrast, the stateless fixed variant remains competitive on depth but does not match the batch-aware variants on the strongest metrics. The ablation therefore supports a narrow design conclusion: the batch-aware chart is the critical component, while the exact initialization rule mainly changes which task trade-off is favored among otherwise stable runs.

A separate ablation isolates the split stop-gradient while keeping the bounded, batch-aware chart and initialization fixed (Appendix E). Removing it changes the learned uncertainty dynamics: $\theta_{\max}$ grows from 3.11 to 3.99 rather than decaying from 3.08 to 2.44, and its final seed-to-seed standard deviation is 0.192 rather than 0.014. Segmentation mIoU also falls from 0.198 to 0.190. The split is therefore a substantive design choice rather than an implementation detail.

Table 4: Real-data transfer benchmarks. Final-epoch task metrics reported as mean ± std over 3 seeds. Higher is better for Yeast metrics and RF1 R^2; lower is better for RF1 RMSE and MAE.

Method	Macro-F1 ↑	Micro-F1 ↑	Hamming Acc ↑	R^2 ↑	RMSE ↓	MAE ↓
Static	0.430 ± 0.008	$\underline{0.610 \pm 0.009}$	0.772 ± 0.003	-0.453 ± 0.127	29.875 ± 0.485	23.376 ± 0.451
Kendall	$\underline{0.431 \pm 0.003}$	0.609 ± 0.001	0.768 ± 0.002	-0.439 ± 0.164	$\underline{29.801 \pm 0.821}$	$\underline{23.219 \pm 0.790}$
UWSO	0.142 ± 0.014	0.486 ± 0.008	0.767 ± 0.002	$\mathbf{-0.408 \pm 0.193}$	30.144 ± 1.137	23.449 ± 1.230
PCGrad	0.430 ± 0.007	0.604 ± 0.010	0.769 ± 0.002	-0.432 ± 0.107	$\mathbf{29.596 \pm 0.385}$	$\mathbf{23.098 \pm 0.368}$
GradNormProxy	0.408 ± 0.028	0.597 ± 0.014	$\underline{0.772 \pm 0.001}$	-0.508 ± 0.248	30.033 ± 0.482	23.380 ± 0.582
BPGS	$\mathbf{0.436 \pm 0.006}$	$\mathbf{0.616 \pm 0.008}$	$\mathbf{0.773 \pm 0.003}$	$\underline{-0.429 \pm 0.213}$	30.158 ± 0.952	23.548 ± 0.908

5.4 Sensitivity to batch statistics and calibration

The batch-aware chart was stable over the tested batch sizes. Across batch sizes 4, 8, and 16, the seed-averaged validation metrics have coefficients of variation below 3.5%, with most below 1.2%; final $\theta_{\max}$ changes modestly from 2.40 to 2.59 (Appendix E). When only the first loader batch is changed, all reported task metrics have coefficients of variation below 2%, and the final $\theta_{\max}$ values differ by at most 0.3%. These studies bound the claim to the tested ranges; they do not establish insensitivity to arbitrary batch statistics.

5.5 Transfer to Yeast and RF1

The real-data benchmarks are less separable than the synthetic stress tests, so we interpret them as evidence of competitiveness rather than universal dominance. On Yeast, BPGS is best on all three reported metrics, with macro-F1 0.436, micro-F1 0.616, and Hamming accuracy 0.773 (Table 4; Appendix Figure 12). On RF1, BPGS is not the top method: PCGrad gives the lowest RMSE and MAE, and UWSO gives the best mean R^2. BPGS nonetheless remains competitive on RF1, with $R^2 = -0.429$, RMSE 30.158, and MAE 23.548 (Appendix Figure 13).

5.6 Computational overhead

On the NYUv2 50% subset, BPGS takes 40.979 seconds per epoch versus 40.916 for Kendall and uses 3368 MB peak GPU memory versus 3339 MB: increases of 0.15% and 0.87%, respectively (Appendix F). The method adds three task-level parameters to an 18.9M-parameter network. Under this controlled setting, the batch statistics and separate uncertainty update do not impose a material efficiency cost.

Taken together, these results bound the scope of the paper's empirical claim. BPGS is strongest where loss-scale mismatch is the central difficulty, remains strong on NYUv2, and stays competitive on two additional real-data multi-output problems. The results do not support a stronger statement that BPGS is uniformly best across all benchmarks and metrics.

6 Discussion and Limitations

The empirical case for BPGS is strongest on the scale-mismatch questions the method was designed to address. The rescaling study isolates numerical-scale invariance directly, the scale-stress study tests degradation under worsening imbalance, and the NYUv2 results show that the same parameterization remains competitive on a standard dense-prediction benchmark. The paper therefore supports a targeted claim about robustness-oriented uncertainty weighting rather than a general claim of dominance over all multi-task optimizers.

The study also has clear limits. First, the benchmark set is small: one dense-prediction benchmark, two tabular real-data problems, and controlled stress tests. A second dense-prediction benchmark remains future work. Second, most headline results use three seeds; the 10-seed scale-stress study strengthens one comparison but does not justify fine-grained ranking claims. Third, BPGS trails Kendall and PCGrad in the noisy and conflict mixed-stress regimes. It stabilizes scalar weights but does not alter gradient directions, so conflict-aware methods can have a structural advantage when gradient interference is the dominant difficulty. Combining BPGS with such methods is untested future work.

Fourth, boundedness does not guarantee learnability at the chart boundary. The logged runs remain inside the boundary (the largest observed $|z_i|/\tau_T$ is 0.93), but this is an empirical margin, not a guarantee for new tasks or training regimes. Fifth, the batch-size and first-batch studies cover only batch sizes 4–16 and the tested NYUv2 subset. Sixth, the formal results establish batch-conditional boundedness and normalized-weight invariance, not convergence or scale-independent optimality. Finally, Nash-MTL is the only added recent baseline; CAGrad, IMTL-G, FAMO, and Auto-Lambda [15] remain outside the evaluated comparison set. The runtime study quantifies cost only on the controlled NYUv2 subset and one GPU setting.

Potential societal impact is indirect. A more stable task-weighting rule can improve the reliability of multi-output models used in scientific or engineering pipelines, but any downstream deployment still inherits the data, labeling, and evaluation failures of the underlying tasks. In safety-sensitive settings, a method that improves one task while weakening another can be misused if only aggregate performance is reported, which is why this paper reports per-task metrics and states where BPGS is not the best method.

7 Conclusion

We presented Bounded Precision-Geometry Scaling (BPGS), a bounded and batch-aware uncertainty-weighting method for multi-task learning under loss-scale mismatch. The normalized weights are exactly invariant to uniform loss rescaling when the numerical safeguards are inactive, and the experiments show that this property translates into stable behavior under pure rescaling and stronger degradation resistance under synthetic scale stress. On NYUv2, BPGS provides the strongest depth metrics and lowest total loss without dominating every task; on Yeast and RF1, it remains competitive beyond the synthetic stress setting.

The evidence supports a targeted conclusion rather than a universal ranking: BPGS is most useful when robustness to heterogeneous loss scales is the primary concern. Its behavior under gradient conflict, broader dense-prediction benchmarks, and combinations with conflict-aware optimizers remain open directions.

References

[1] Rich Caruana. Multitask learning. *Machine Learning*, 28(1):41–75, 1997. doi: 10.1023/A:1007379606734.

[2] Alex Kendall, Yarin Gal, and Roberto Cipolla. Multi-task learning using uncertainty to weigh losses for scene geometry and semantics. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 7482–7491, 2018.

[3] Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient surgery for multi-task learning. In *Advances in Neural Information Processing Systems*, 2020.

[4] Zhao Chen, Vijay Badrinarayanan, Chen-Yu Lee, and Andrew Rabinovich. Gradnorm: Gradient normalization for adaptive loss balancing in deep multitask networks. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 794–803, 2018.

[5] Michelle Guo, Albert Haque, De-An Huang, Serena Yeung, and Li Fei-Fei. Dynamic task prioritization for multitask learning. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 282–299, 2018.

[6] Ozan Sener and Vladlen Koltun. Multi-task learning as multi-objective optimization. In *Advances in Neural Information Processing Systems*, 2018.

[7] Bo Liu, Xingchao Liu, Xiaojie Jin, Peter Stone, and Qiang Liu. Conflict-averse gradient descent for multi-task learning. In *Advances in Neural Information Processing Systems*, 2021.

[8] Aviv Navon, Aviv Shamsian, Idan Achituve, Haggai Maron, Kenji Kawaguchi, Gal Chechik, and Ethan Fetaya. Multi-task learning as a bargaining game. In *Proceedings of the 39th International*

Conference on Machine Learning, volume 162 of *Proceedings of Machine Learning Research*, pages 16428–16446, 2022.

[9] Liyang Liu, Yanqi Li, Andrew J. Davison, and Edward Johns. Independent component alignment for multi-task learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 2008–2017, 2021.

[10] Bo Liu, Yihua Feng, Chrisantha Fernando, Cesar Rodríguez-Opazo, Ian Reid, and Stephen Gould. FAMO: Fast adaptive multitask optimization. In *Advances in Neural Information Processing Systems*, 2023.

[11] Nathan Silberman, Derek Hoiem, Pushmeet Kohli, and Rob Fergus. Indoor segmentation and support inference from RGBD images. In *European Conference on Computer Vision*, 2012.

[12] André Elisseeff and Jason Weston. A kernel method for multi-labelled classification. In *Advances in Neural Information Processing Systems*, 2001.

[13] Eleftherios Spyromitros-Xioufis, Grigorios Tsoumakas, William Groves, and Ioannis Vlahavas. Multi-target regression via input space expansion: treating targets as inputs. *Machine Learning*, 104(1):55–98, 2016. doi: 10.1007/s10994-016-5546-z.

[14] Shikun Liu, Edward Johns, and Andrew J. Davison. End-to-end multi-task learning with attention. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019.

[15] Shikun Liu, Stephen James, Andrew J. Davison, and Edward Johns. Auto-lambda: Disentangling dynamic task relationships. *Transactions on Machine Learning Research*, 2022.

A Reproducibility Details

Unless stated otherwise, main-paper tables aggregate over seeds 42, 43, and 44. The synthetic scale-stress table uses 10 seeds, as stated in its caption. The stress studies use per-seed `results.csv` summaries. NYUv2, Yeast, and RF1 tables are extracted from epoch-level `metrics.csv` logs using the final recorded epoch for each run. This final-epoch rule is especially important for NYUv2 because checkpoint metadata is not consistently available across methods.

Optimizer fidelity. The batch log-loss statistics $\mu(L)$ and $\bar{\varsigma}(L)$ are detached from the network computation graph. Network training uses detached normalized weights, while the uncertainty update uses $g_\theta = 100$ in every reported BPGS configuration. BPGS uses gradient clipping 10.0; the Kendall ablation baseline uses 1.0, matching its method configuration.

Main NYUv2 benchmark. The standard NYUv2 benchmark uses 795 training images and 654 validation images with semantic segmentation, depth estimation, and surface-normal prediction. Representative BPGS runs use 120 epochs, batch size 8, learning rate 5×10^{-4}, minimum learning rate 10^{-6}, 100 warmup steps, weight decay 10^{-4}, gradient clipping 10.0, deterministic execution with CPU batch augmentation, and mixed-precision training. Checkpoint selection is done with `val/miou` in max mode for the final benchmark runs, as specified in the study overrides. The configuration files include early-stopping fields, but the paper tables are extracted from the final epoch for every method.

NYUv2 ablation. The ablation uses a fixed 50% training subset (398 of 795 images, selected via stratified sampling by majority semantic class and depth quartile using seed 11) with the same validation split. Representative runs use 60 epochs, batch size 8, learning rate 5×10^{-4}, minimum learning rate 10^{-6}, 100 warmup steps, weight decay 10^{-4}, mixed precision, and the dataset default selection rule (`val/total_loss` in min mode). The Kendall baseline uses gradient clipping 1.0, while the BPGS variants use gradient clipping 10.0, matching their respective method configurations.

Yeast and RF1. Yeast uses 1,500 training examples and 917 validation examples; RF1 uses 4,108 training examples and 5,017 validation examples. Representative Yeast BPGS runs use 120 epochs, batch size 128, learning rate 5×10^{-4}, minimum learning rate 10^{-5}, 200 warmup steps, weight decay 10^{-4}, gradient clipping 10.0, and selection metric `val/total_loss` with patience 15. Representative RF1 BPGS runs use the same optimizer settings, batch size 256, and the same selection rule.

Table 5: Synthetic scale stress: Macro Score across scale factors. Mean ± std over 10 seeds.

Method	$\times 1$	$\times 10$	$\times 100$	$\times 1000$
Kendall	0.738 ± 0.017	0.607 ± 0.038	0.522 ± 0.030	0.499 ± 0.025
UWSO	0.690 ± 0.034	0.604 ± 0.040	0.525 ± 0.035	0.494 ± 0.024
BPGS	$\mathbf{0.743 \pm 0.018}$	$\mathbf{0.616 \pm 0.039}$	$\mathbf{0.547 \pm 0.032}$	$\mathbf{0.543 \pm 0.032}$

Table 6: Relative Macro Score change (%) from $\times 1 \rightarrow \times 1000$. Positive values indicate degradation; negative values indicate improvement.

Method	Scale Stress	Loss Rescaling
Kendall	32.3%	18.4%
UWSO	28.5%	$\mathbf{-9.5}\%$
BPGS	$\mathbf{27.0}\%$	0.0%

Method naming. The real-data comparison includes `GradNormProxy`, which is a gradient-norm proxy baseline inspired by GradNorm rather than a claim of faithful reimplementation. The UWSO baseline is an analytical inverse-loss weighting rule with fixed temperature as implemented in this codebase.

Additional NYUv2 studies. The stop-gradient, batch-size, first-batch, and overhead studies all use the 50% NYUv2 subset and 60 epochs. The first three use the settings described in the main text; the overhead study compares BPGS and Kendall over seeds 42, 43, and 44. Nash-MTL uses the main NYUv2 120-epoch protocol with batch size 8, AdamW learning rate 5×10^{-4}, weight decay 10^{-4}, 100 warmup steps, cosine decay to 10^{-6}, mixed precision, and gradient clipping 1.0. The normalization ablation uses the pure-rescaling grid and seeds 42, 123, and 999.

Submission-scope disclosures. The experiments use standard public benchmarks cited in the main text. The anonymized submission materials include the implementation, configuration files, and reproduction scripts. Uniform hardware and wall-clock records for every archived run are not included; the dedicated overhead study is therefore limited to the controlled setting reported in the paper.

B Additional Stress Results

C Ablation and NYUv2 Figures

Nash-MTL's per-step coefficients varied substantially across batches in all three runs. This observation is descriptive only: the present logs do not distinguish solver variation from an intrinsic property of the method on this benchmark.

D Real-Data Training Curves

E Sensitivity Analyses

The stop-gradient study changes only the coupling of the uncertainty update. The batch-size and first-batch studies are controlled diagnostics on the NYUv2 subset, not claims about all dataset sizes, batch regimes, or initialization procedures.

F Computational Overhead

The measurements report the controlled BPGS–Kendall comparison on a single GPU setting. They quantify the added batch-statistics computation and uncertainty pass, but do not establish a hardware-independent efficiency guarantee.

Table 7: Heterogeneous mixed stress: Macro Score and Worst-Task Score across regimes. Mean ± std over 3 seeds.

Method	Clean		Noisy		Conflict	
	Macro	Worst	Macro	Worst	Macro	Worst
Kendall	0.687 ± 0.010	0.267 ± 0.056	$\mathbf{0.659 \pm 0.007}$	$\mathbf{0.242 \pm 0.045}$	$\mathbf{0.664 \pm 0.012}$	$\mathbf{0.211 \pm 0.041}$
UWSO	0.437 ± 0.028	0.000 ± 0.000	0.438 ± 0.020	0.000 ± 0.000	0.433 ± 0.013	0.000 ± 0.000
PCGrad	$\mathbf{0.688 \pm 0.016}$	0.274 ± 0.055	0.656 ± 0.007	0.241 ± 0.046	0.660 ± 0.011	0.205 ± 0.048
BPGS	0.679 ± 0.017	$\mathbf{0.301 \pm 0.052}$	0.641 ± 0.013	0.176 ± 0.076	0.652 ± 0.012	0.198 ± 0.063

Table 8: Normalization ablation on pure loss rescaling. Macro score (mean ± std over seeds 42, 123, and 999). L1-normalizing Kendall improves the largest-scale result but does not reproduce the BPGS scale sensitivity.

Scale	BPGS	Kendall	Kendall+L1
$\times 1$	0.789 ± 0.018	0.788 ± 0.014	0.794 ± 0.017
$\times 10$	0.791 ± 0.015	0.781 ± 0.023	0.780 ± 0.022
$\times 100$	0.788 ± 0.016	0.734 ± 0.037	0.742 ± 0.029
$\times 1000$	0.785 ± 0.017	0.658 ± 0.042	0.689 ± 0.034
$\Delta(\times 1 \to \times 1000)$	-0.004	-0.130	-0.105

G Bounded-Chart Behavior

We reconstruct $z_i = (s_i - \mu)/\bar{\varsigma}$ from the logged batch statistics and log-variances. At the closest observed point, $\sigma(\theta_i)(1 - \sigma(\theta_i)) \approx 0.033$, about 13% of its maximum 0.25; by the final epochs the corresponding value is approximately 0.074. These measurements show an empirical margin from saturation in the logged runs, not a general guarantee.

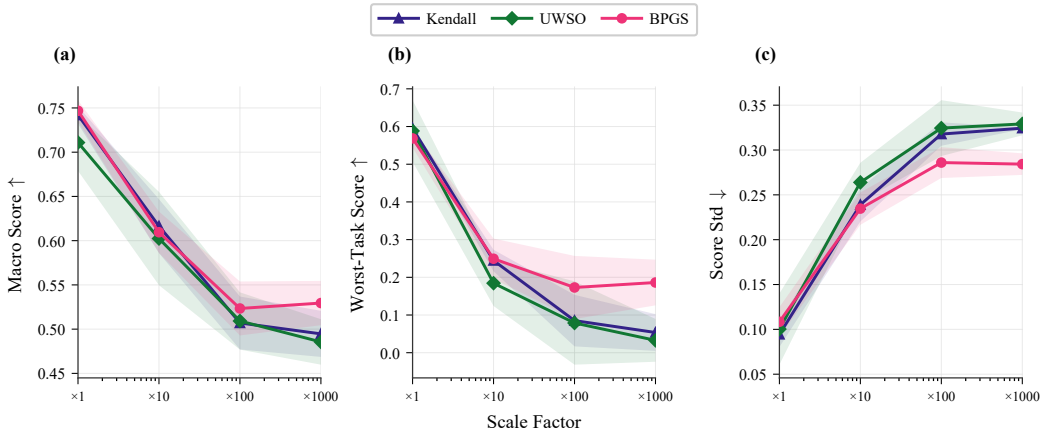

Figure 2: Synthetic scale-stress curves. As the scale factor increases, all methods degrade, but BPGS retains the strongest macro score at every tested factor and tracks the strongest worst-task score at the larger stress levels.

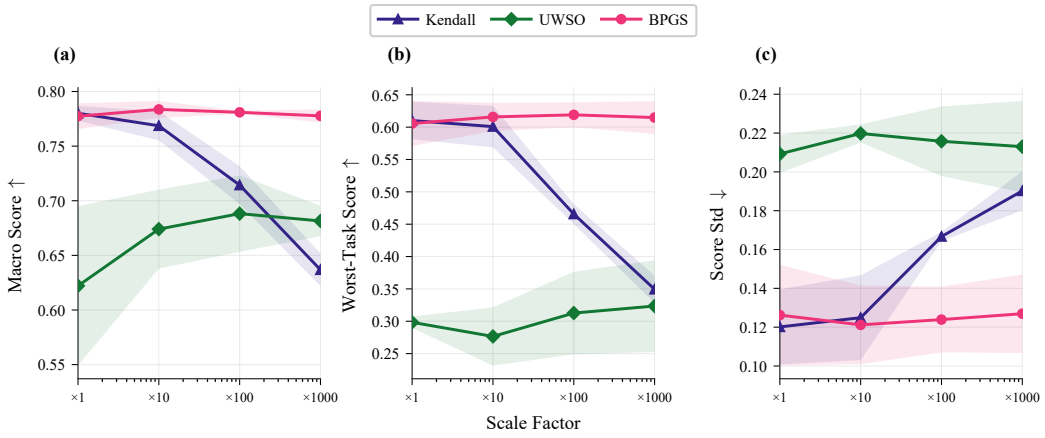

Figure 3: Pure loss-rescaling curves. BPGS remains nearly invariant across the entire rescaling range, while Kendall degrades sharply at larger factors and UWSO improves only from a substantially weaker starting point.

Table 9: Stop-gradient ablation on the NYUv2 50% subset (three seeds, 60 epochs). Removing the stop-gradient changes the uncertainty dynamics and lowers segmentation mIoU.

Metric	Split on	Split off
$\theta_{\max}$, epoch 0	3.082 ± 0.017	3.109 ± 0.011
$\theta_{\max}$, final	2.438 ± 0.014	3.993 ± 0.192
Segmentation mIoU	0.1983	0.1897
Depth RMSE	0.8433	0.8544
Total loss	2.1278	2.1419

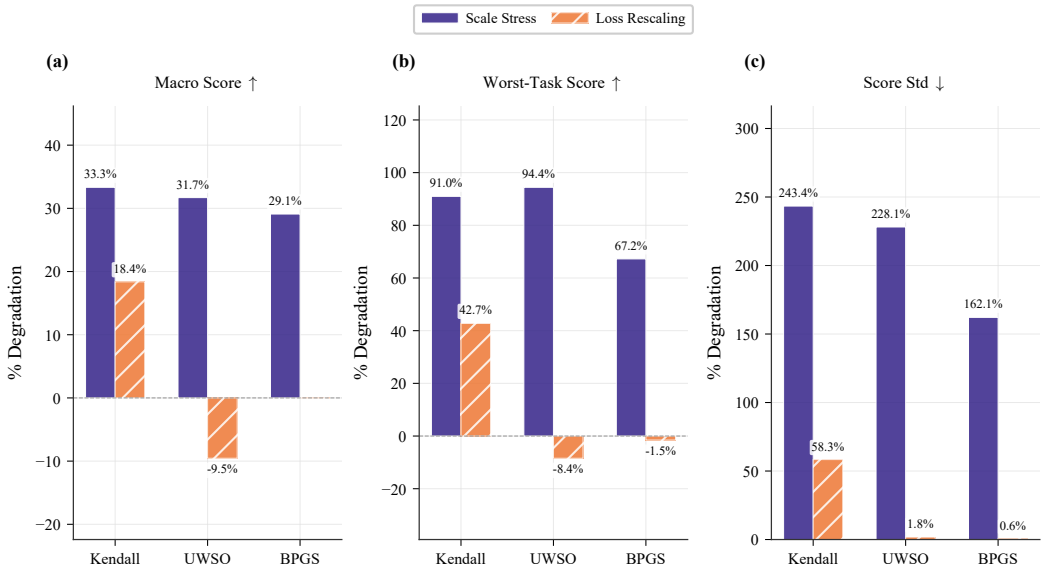

Figure 4: Relative degradation comparison between synthetic scale stress and pure loss rescaling. The figure shows that BPGS has the smallest macro-score degradation under scale stress and essentially no macro-score degradation under pure rescaling.

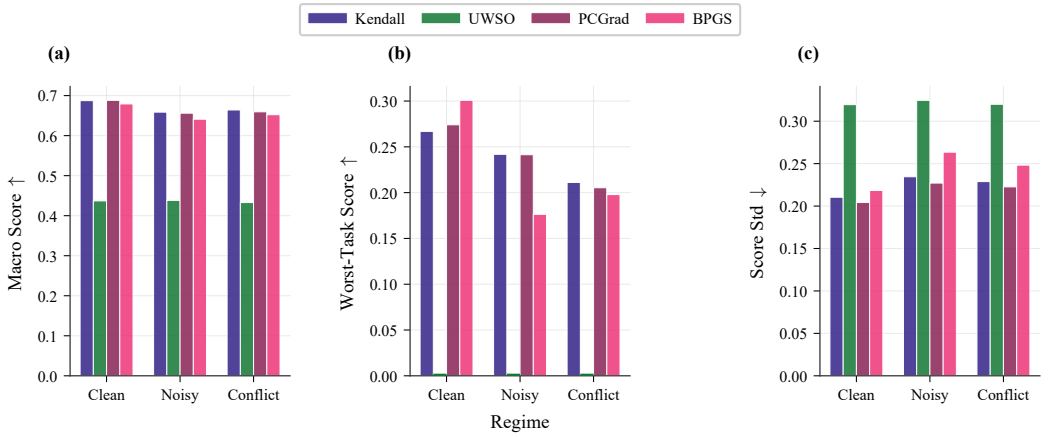

Figure 5: Heterogeneous mixed-stress summary. BPGS is not the best method in every regime, but it achieves the strongest worst-task score in the clean regime and remains well above the degenerate UWSO worst-task score across all regimes.

Table 10: Batch-size sensitivity of canonical BPGS on the NYUv2 50% subset (three seeds per batch size, 60 epochs). Values are seed-averaged final metrics.

Metric	bs=4	bs=8	bs=16	CV
Total loss	2.1468	2.1226	2.1507	0.58%
Depth Abs Rel	0.2411	0.2421	0.2451	0.70%
Depth RMSE	0.8643	0.8403	0.8490	1.17%
Normal angle	29.400	29.223	29.754	0.75%
Segmentation mIoU	0.1955	0.1932	0.1810	3.35%
$\theta_{\max}$, final	2.396	2.441	2.589	–

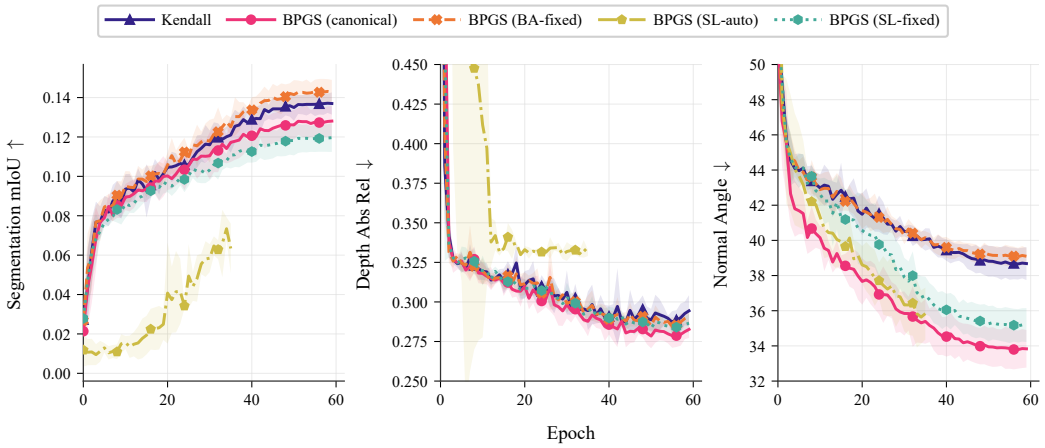

Figure 6: NYUv2 ablation validation curves by task. The stateless auto-calibrated variant fails on segmentation and remains weak on depth, while the batch-aware variants maintain stronger trajectories throughout training.

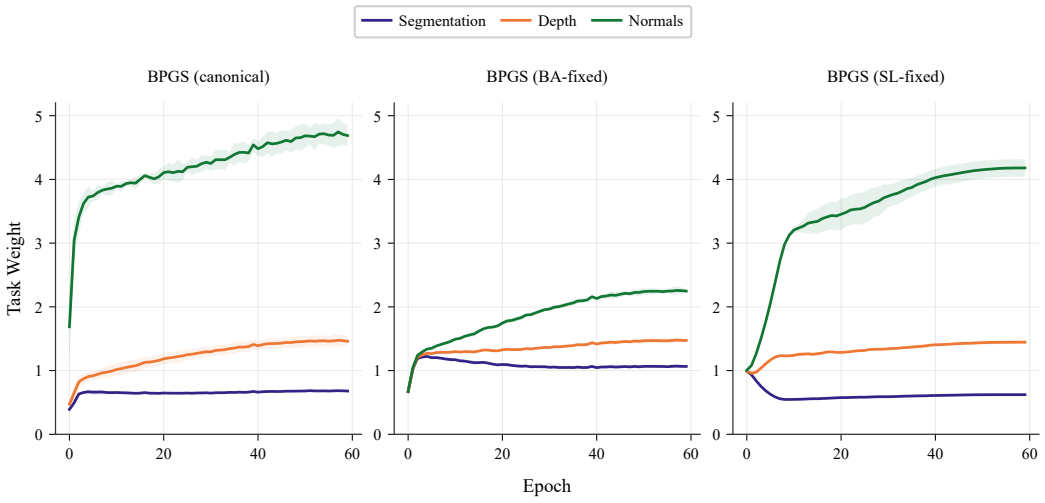

Figure 7: Task-weight dynamics for representative stable BPGS variants on the NYUv2 ablation. The batch-aware variants keep task weights in distinct but controlled regimes, whereas the stateless fixed variant shows a different long-run allocation pattern, especially for the surface-normal task.

Table 11: First-batch calibration sensitivity on the NYUv2 50% subset. Only the loader seed changes; the model seed is fixed.

Metric	1001	2001	3001	CV
Total loss	2.1145	2.1040	2.1090	0.20%
Depth Abs Rel	0.2385	0.2392	0.2411	0.46%
Depth RMSE	0.8348	0.8302	0.8325	0.23%
Normal angle	28.9205	28.8975	29.1069	0.32%
Segmentation mIoU	0.1912	0.1866	0.1952	1.84%
$\theta_{\max}$, final	2.4451	2.4382	2.4380	0.3% spread

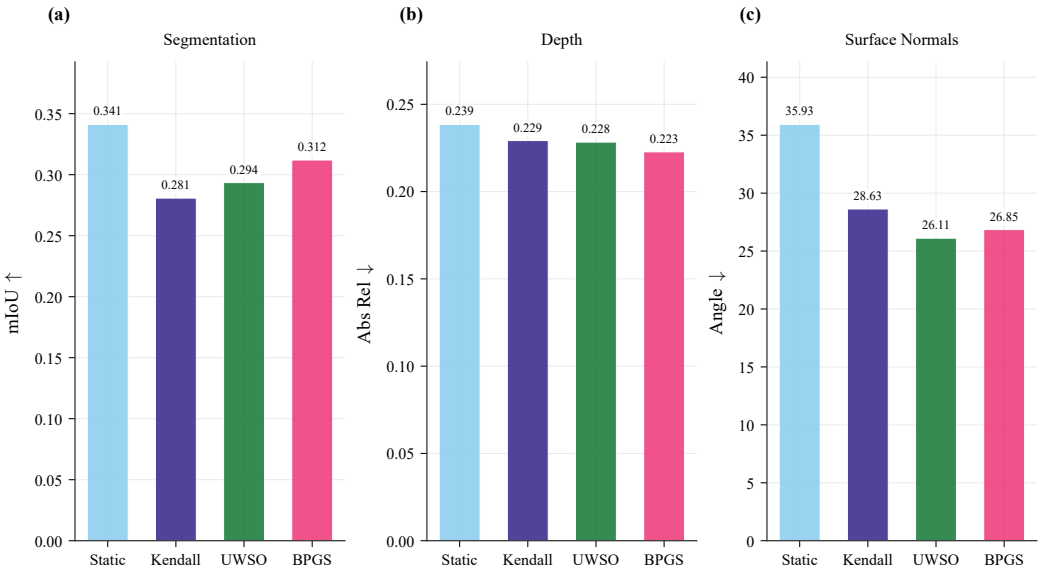

Figure 8: NYUv2 per-task performance comparison. The figure makes the trade-off pattern visually explicit: Static is strongest on segmentation, UWSO is strongest on surface normals, and BPGS is strongest on both depth metrics.

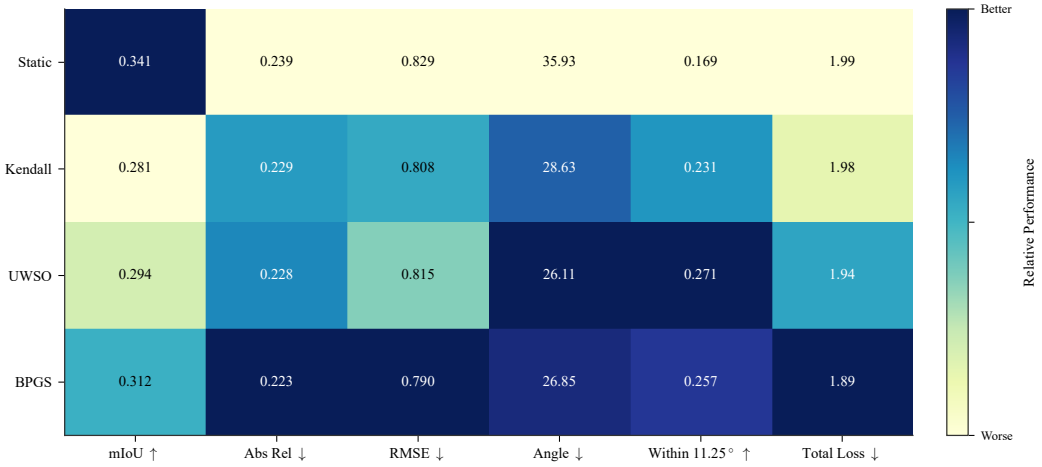

Figure 9: NYUv2 performance heatmap across reported metrics. Darker cells indicate stronger relative performance after accounting for metric direction. BPGS concentrates its gains on depth and total loss, while Static and UWSO remain stronger on some other tasks.

Table 12: Computational overhead on the NYUv2 50% subset (three seeds, 60 epochs).

Quantity	BPGS	Kendall	Difference
Per-epoch time (s)	40.979	40.916	+0.15%
Peak GPU memory (MB)	3368.36	3339.34	+0.87%
Total time, 60 epochs (s)	2461.6	2457.9	+3.7
Trainable parameters	18,869,143	18,869,140	+3

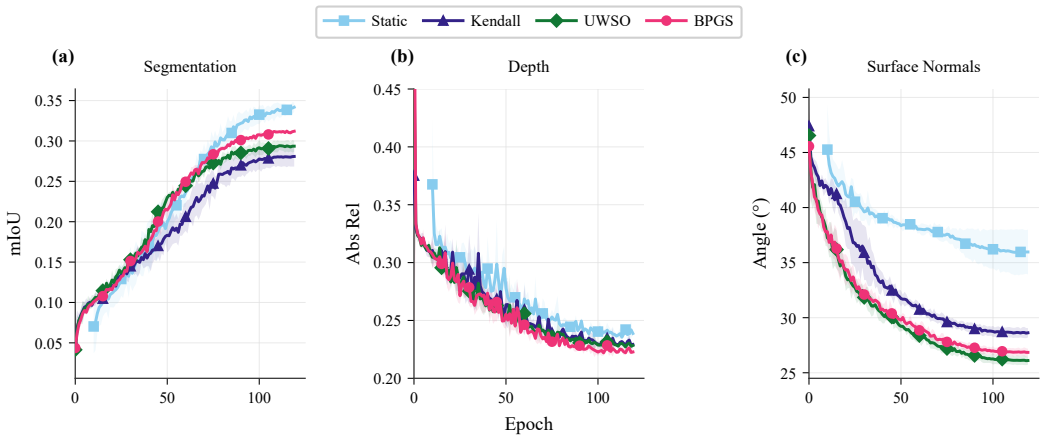

Figure 10: NYUv2 validation training curves by task. BPGS tracks the strongest depth trajectory and remains competitive on segmentation and surface normals over the full 120-epoch budget.

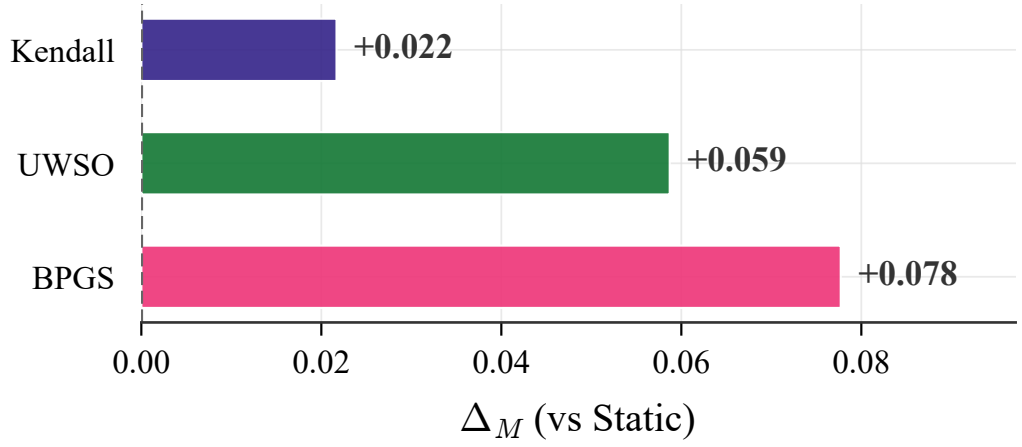

Figure 11: NYUv2 Δ_M comparison against the Static baseline. BPGS attains the strongest aggregate improvement among the adaptive methods evaluated in the main benchmark.

Table 13: Closest observed approach to the bounded-chart radius across logged main-paper runs. The maximum occurs at initialization; no run reaches the boundary.

Dataset	T	max $\lvert z_i \rvert / \tau_T$	final cap
NYUv2	3	0.9327	≤ 0.84
Yeast	14	0.7017	≤ 0.69
RF1	8	0.8937	≤ 0.89

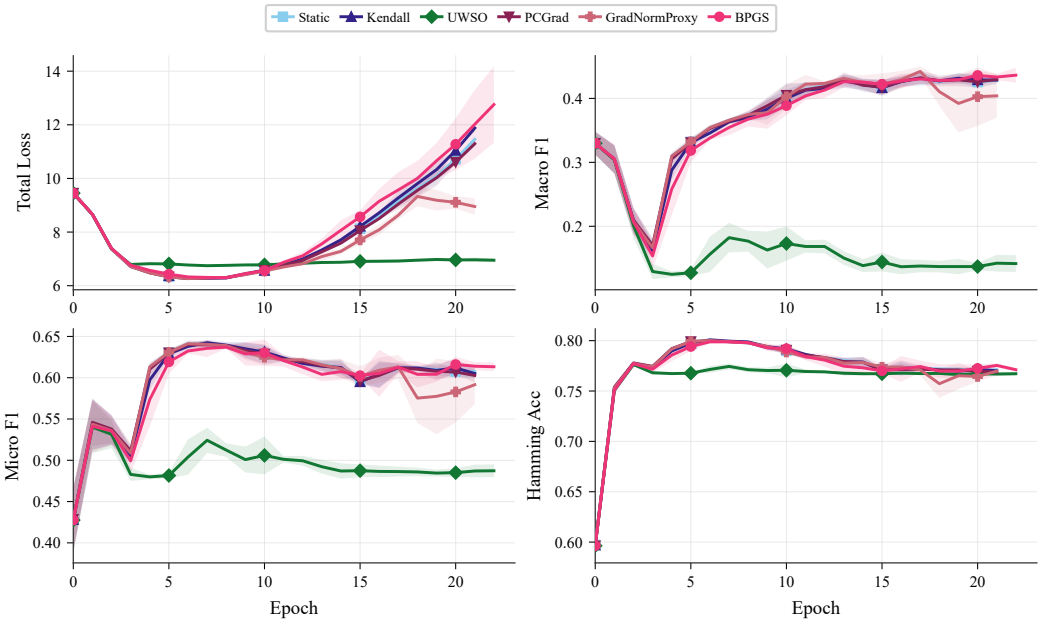

Figure 12: Yeast training curves. BPGS tracks the strongest final micro-F1 and macro-F1 among the compared methods, while UWSO remains substantially weaker throughout training.

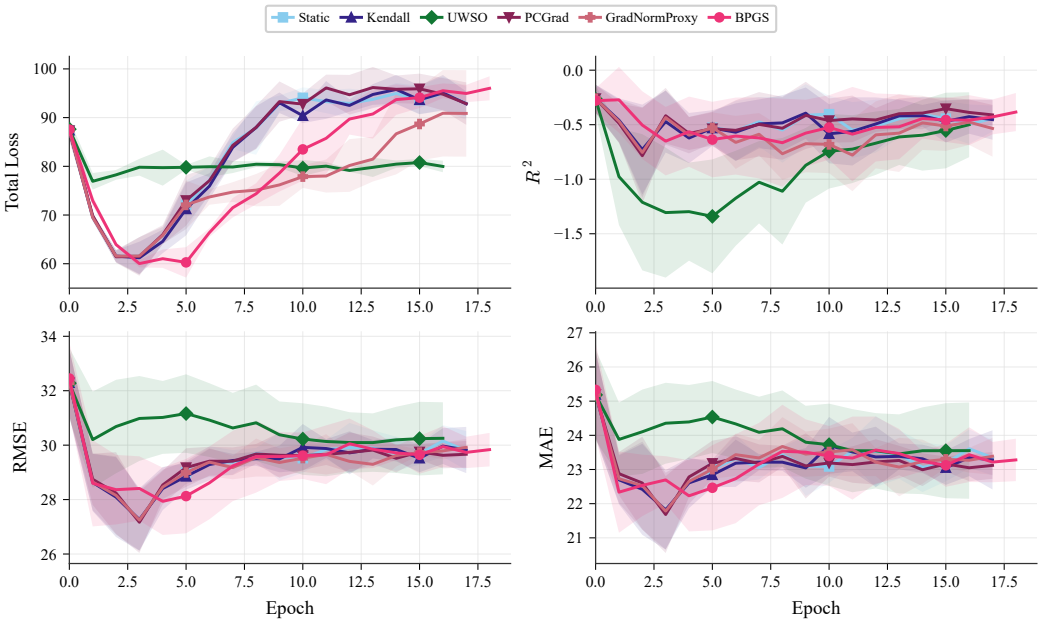

Figure 13: RF1 training curves. The methods remain closer than in the synthetic stress studies, which is consistent with the paper's claim that RF1 supports competitiveness rather than clear superiority.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction state the contribution as robustness under loss-scale mismatch and explicitly avoid a claim of universal superiority; see the abstract and Section 1.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 6 discusses benchmark scope, seed count, the absence of global optimization guarantees, and the dependence on batch statistics.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: Section 3.4 states the batch-conditional boundedness result, its assumptions, and the complete proof argument based on the bounded sigmoid chart.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: Sections 3–5 and Appendix A specify the datasets, tasks, seeds, extraction rule, budgets, principal hyperparameters, and compared methods that support the paper's claims.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
   - While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
     (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
     (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
     (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
     (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: The anonymized supplementary materials contain the implementation, configuration files, and scripts to reproduce the main experimental results. The datasets used are standard public benchmarks cited in the main text.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Section 4 states the benchmark protocols and reporting rules, and Appendix A provides the representative training budgets and hyperparameters used for the main paper assets.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: The tables and figures supporting the main claims report mean $\pm$ standard deviation across seeds, and Section 4 states that the variability is computed over seeds 42, 43, and 44.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.

- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [No]

   Justification: The submission reports training budgets and precision settings but does not provide complete hardware specifications or total compute accounting for every experiment.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The work is a methodological study on task weighting over standard benchmarks, does not involve human subjects, and does not raise an exception to the NeurIPS Code of Ethics.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: Section 6 includes a brief broader-impact discussion covering both possible reliability benefits and the risk of hiding per-task failures behind aggregate scores.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The submission does not release a high-risk model or dataset and does not study a misuse-sensitive generative system.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [No]

    Justification: The submission cites the benchmark datasets and related methods, but it does not provide a complete per-asset license and terms-of-use audit in the paper.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.

- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [N/A]

    Justification: The submission does not introduce and publicly release a new dataset, benchmark, or model asset.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: The work does not involve crowdsourcing or research with human subjects.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: The work does not involve human subjects, so IRB review is not applicable.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.

- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [N/A]

    Justification: LLMs are not part of the proposed method, experiments, or scientific claims.

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.